

SENAC RJ — Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

SPRINT 6 — INTEGRAÇÃO COM O ECOSSISTEMA ÁGIL

Tema: DDD — Entendendo o Domínio do Problema

Entrega: 19/06/2026

Este documento demonstra como o Domain-Driven Design se integra a um contexto ágil de desenvolvimento, mostrando como o modelo de domínio construído nas sprints anteriores guia diretamente a construção do **Product Backlog** e a redação de **histórias de usuário (User Stories)** do sistema fictício de Gestão de Entregas Logísticas.

1. DDD e Agilidade: Uma Relação Complementar

Embora o DDD não seja, por si só, uma metodologia ágil, ele se integra naturalmente a frameworks como Scrum e Kanban porque resolve um problema que métodos ágeis tradicionais não respondem por completo: **o que** deve ser construído. Enquanto o Scrum organiza o processo de entrega (sprints, papéis, cerimônias), o DDD fornece o vocabulário e a estrutura conceitual que dá substância ao backlog.

A ideia central desta sprint é mostrar que o **modelo de domínio não é um artefato paralelo à agilidade — ele é o insumo principal para transformar necessidades de negócio em itens de backlog rastreáveis e em histórias de usuário com critérios de aceite precisos**, evitando a ambiguidade que normalmente surge quando o Product Owner e a equipe técnica usam vocabulários diferentes.

2. Do Modelo de Domínio ao Product Backlog

Cada elemento do modelo tático construído na Sprint 4 — Agregados, Eventos de Domínio e Serviços de Domínio — pode ser traduzido em itens estruturados de backlog. A tabela abaixo mostra esse mapeamento direto:

Elemento do Modelo de Domínio	Tipo de Item de Backlog	Exemplo no Sistema de Entregas
Agregado (ex.: Pedido)	Épico	Épico: 'Gestão do ciclo de vida do Pedido'
Comportamento da Entidade Raiz (ex.: criar Pedido, adicionar Item)	História de Usuário	'Como Remetente, quero criar um Pedido com múltiplos itens...'
Invariante de negócio (ex.: Pedido deve ter ao menos 1 Item)	Critério de Aceite	'Dado que não há itens, o sistema deve impedir a criação do Pedido.'
Evento de Domínio (ex.: PedidoCriado)	Gatilho de integração entre histórias / equipes	Define a dependência entre a história de 'criar Pedido' e a de 'agendar Rota'.
Serviço de Domínio (ex.: RoteirizadorDeEntregas)	Feature técnica dentro de uma História	'Como sistema, preciso calcular a rota ótima respeitando capacidade do veículo.'
Bounded Context (ex.: Roteirização)	Tema / Squad responsável	Define qual time do Scrum é dono daquele conjunto de histórias.

Esse mapeamento evita um problema comum em backlogs construídos sem base em domínio: histórias redigidas a partir de telas ou funcionalidades técnicas, sem relação clara com o valor de negócio ou com as regras que precisam ser respeitadas.

3. Diagrama de Integração — Modelo de Domínio Guiando o Backlog

O fluxo abaixo representa como a estrutura DDD da Sprint 4 alimenta o backlog de um Sprint Planning típico no Scrum:

FLUXO DE INTEGRAÇÃO: DDD → BACKLOG → SPRINT

- 1. Domain Discovery** — Event Storming com domain experts identifica Eventos de Domínio e Bounded Contexts (insumo: Linguagem Ubíqua da Sprint 3).
- 2. Modelagem Tática** — Agregados, Entidades e Value Objects são definidos por Bounded Context (insumo: modelagem da Sprint 4).
- 3. Product Backlog Building** — Cada Agregado gera um Épico; cada comportamento da Raiz do Agregado gera uma História de Usuário; cada invariante gera um Critério de Aceite.
- 4. Backlog Refinement** — Histórias são refinadas e estimadas pela equipe, usando a Linguagem Ubíqua para eliminar ambiguidade na conversa com o PO.
- 5. Sprint Planning** — Histórias de um mesmo Bounded Context tendem a ser puxadas pelo mesmo squad, respeitando a fronteira de consistência do agregado.
- 6. Sprint Execution** — Código é escrito usando os mesmos termos do glossário de domínio (classes, métodos, nomes de eventos).

4. Histórias de Usuário Derivadas do Modelo de Domínio

A seguir, exemplos de histórias de usuário escritas diretamente a partir dos Agregados e Eventos de Domínio modelados nas sprints anteriores, cada uma já com critérios de aceite derivados das invariantes de negócio identificadas:

4.1 Bounded Context: Pedidos

HISTÓRIA #1

Como Remetente,
quero criar um Pedido com um ou mais Itens e um Endereço de Entrega,
para que minha mercadoria seja transportada até o Destinatário.

Critérios de aceite (derivados das invariantes do Agregado Pedido):

- O Pedido deve conter ao menos 1 Item (regra do agregado).
- O Endereço de Entrega deve ter CEP válido (regra do Value Object Endereco).
- O Pedido nasce com status CRIADO (regra do Value Object StatusPedido).
- Ao salvar, o evento PedidoCriado deve ser emitido.

4.2 Bounded Context: Roteirização

HISTÓRIA #2

Como Sistema de Roteirização,
quero calcular automaticamente uma Rota ao receber um PedidoCriado,
para que o Motorista saiba a sequência de Paradas a executar.

Critérios de aceite (derivados do Serviço de Domínio RoteirizadorDeEntregas):

- O peso total dos Pedidos roteirizados não pode exceder a capacidadeKg do Veículo.
- A janela de tempo de cada Parada deve ser respeitada na sequência calculada.
- O Motorista alocado não pode exceder 8h de jornada na Rota.
- Ao concluir o cálculo, o evento RotaCalculada deve ser emitido.

4.3 Bounded Context: Rastreamento

HISTÓRIA #3

Como Destinatário,
quero acompanhar o status da minha Remessa em tempo real,
para saber quando minha encomenda chegará.

Critérios de aceite (derivados do Agregado Remessa):

- A Remessa deve ser criada automaticamente ao receber o evento PedidoCriado.
- Cada novo EventoDeRastreamento deve recalcular o ETA exibido ao Destinatário.
- O status da Remessa avança de forma unidirecional (sem retrocessos).

5. Priorização do Backlog Guiada pelo Core Domain

Conforme discutido na análise crítica da Sprint 5, o esforço de modelagem DDD deve se concentrar no Core Domain. Esse mesmo princípio orienta a priorização do backlog: histórias relacionadas ao Core Domain (Roteirização, o diferencial competitivo do sistema) recebem prioridade mais alta do que histórias de Generic Domains (como autenticação ou notificações), que podem inclusive ser resolvidas com soluções de terceiros sem prejuízo ao valor entregue.

Bounded Context	Classificação (Sprint 3)	Prioridade no Backlog	Justificativa
Roteirização	Core Domain	Alta	Onde está a vantagem competitiva (otimização de rotas).
Pedidos	Supporting Domain	Alta	Pré-requisito funcional para o Core Domain operar.
Rastreamento	Supporting Domain	Média	Importante para experiência do cliente, mas não bloqueia o core.
Notificações	Generic Domain	Baixa	Pode usar serviço de terceiros (ex.: Twilio, SendGrid).
Financeiro	Generic Domain	Baixa	Pode integrar com gateway de pagamento já existente no mercado.

6. Considerações Finais

Esta sprint encerra o percurso do grupo sobre DDD demonstrando que a abordagem não é um exercício teórico isolado, mas uma ferramenta que se conecta diretamente à prática ágil: o modelo de domínio constrói a Linguagem Ubíqua (Sprint 3), a Linguagem Ubíqua estrutura o modelo tático (Sprint 4), o modelo tático revela onde o esforço deve ser concentrado (Sprint 5), e essa mesma estrutura, finalmente, organiza e prioriza o Product Backlog (Sprint 6).

Esse encadeamento mostra que DDD e agilidade não competem entre si: o DDD responde à pergunta '*o que estamos construindo e por quê*', enquanto o Scrum e o Kanban respondem à pergunta '*como vamos entregar isso de forma iterativa e incremental*'. Juntas, as duas abordagens reduzem tanto o risco de construir o produto errado quanto o risco de entregá-lo de forma desorganizada.